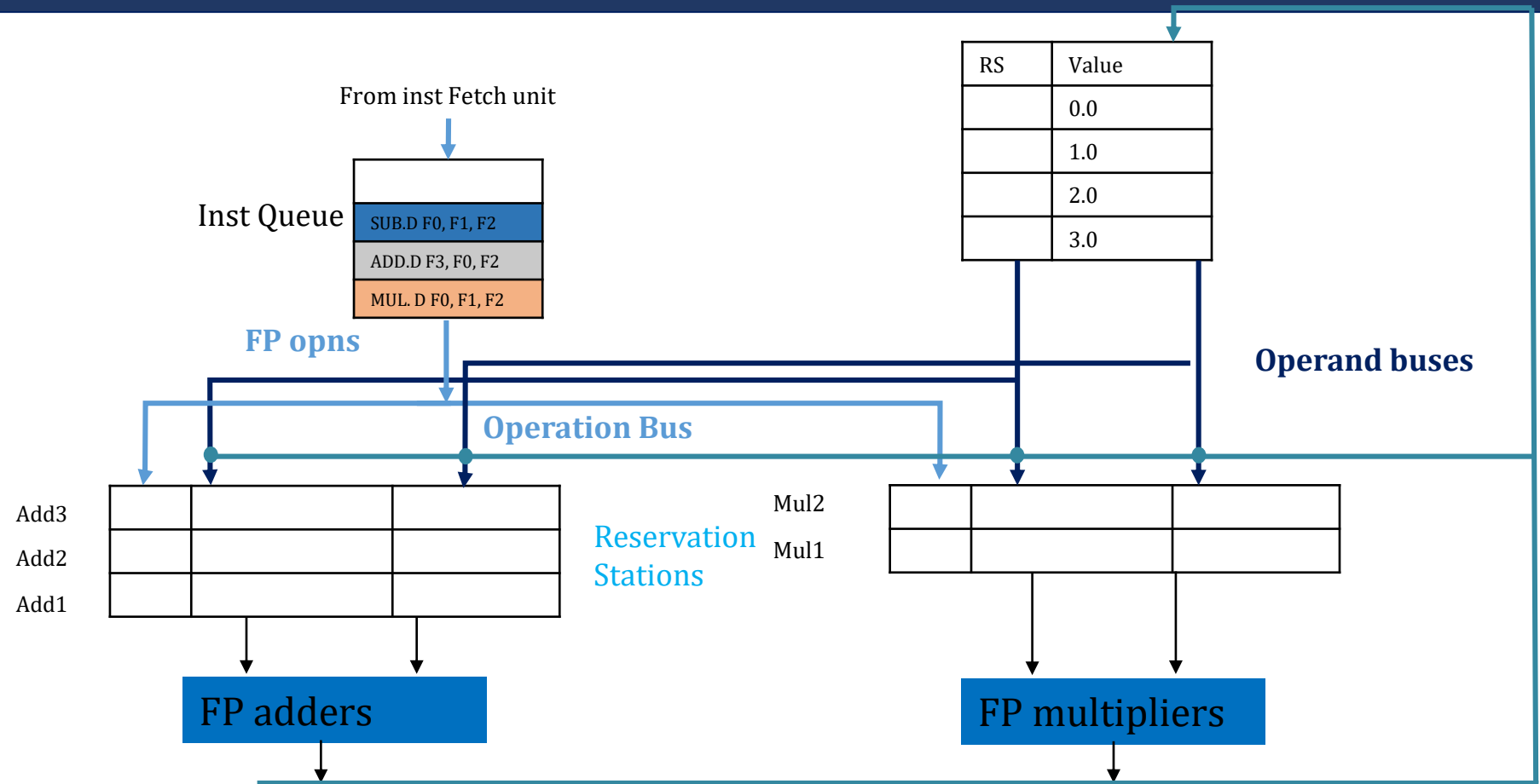


Lecture 4: Dynamic Scheduling

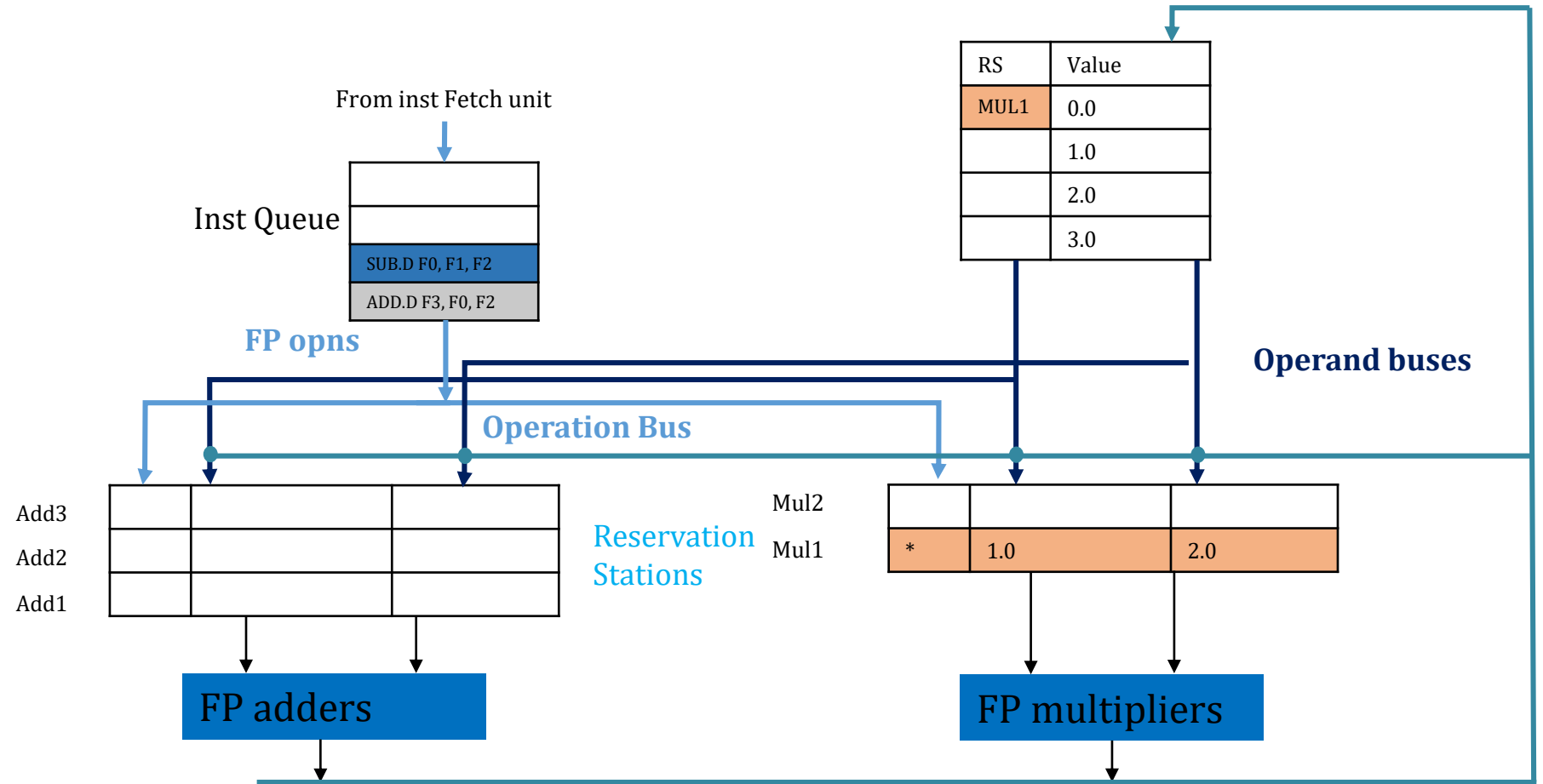
Example

MUL.D F0, F1, F2
ADD.D F3, F0, F2
SUB.D F0, F1, F2



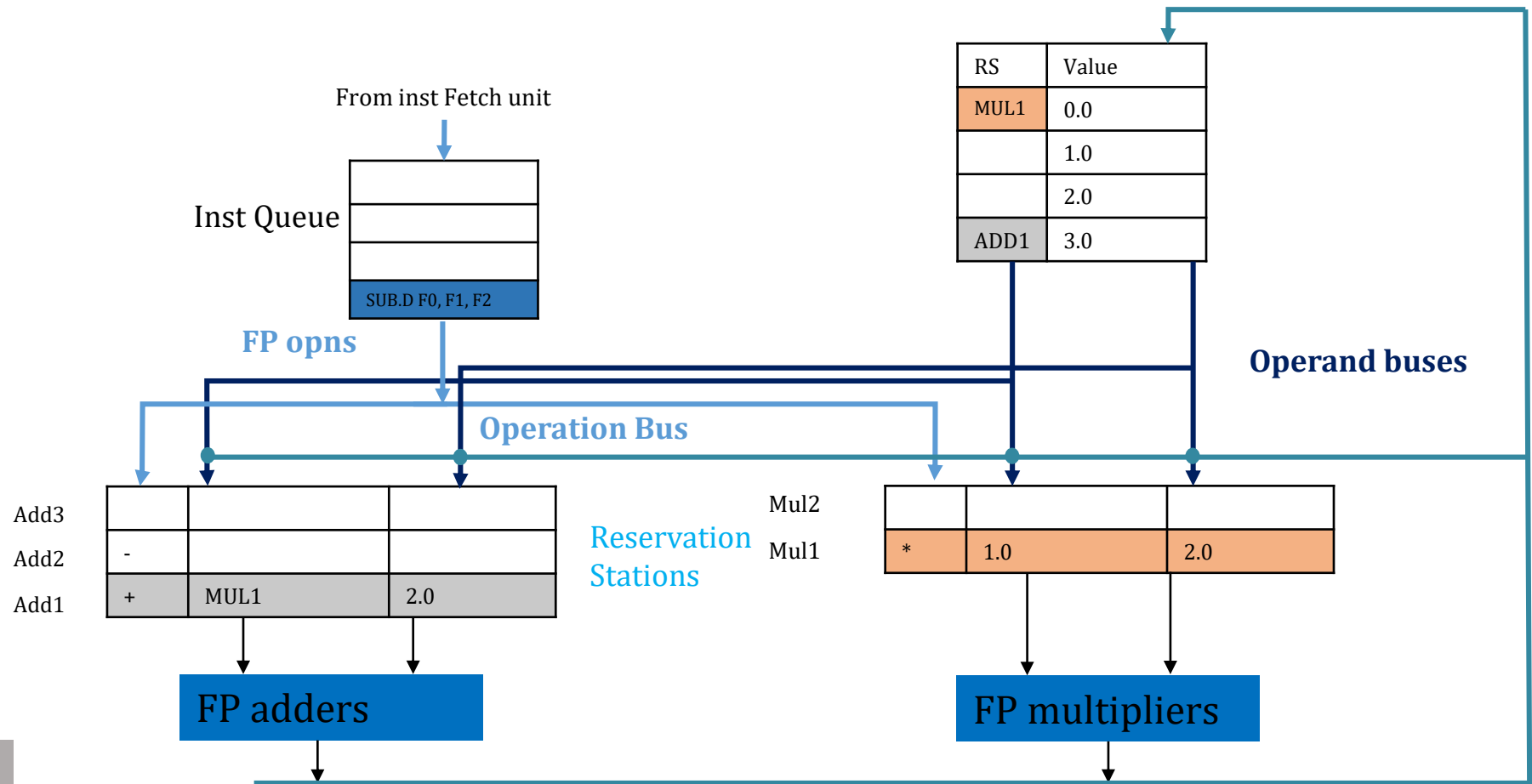
Issue MUL.D

MUL.D F0, F1, F2
ADD.D F3, F0, F2
SUB.D F0, F1, F2



Issue ADD.D

MUL.D F0, F1, F2
ADD.D F3, F0, F2
SUB.D F0, F1, F2

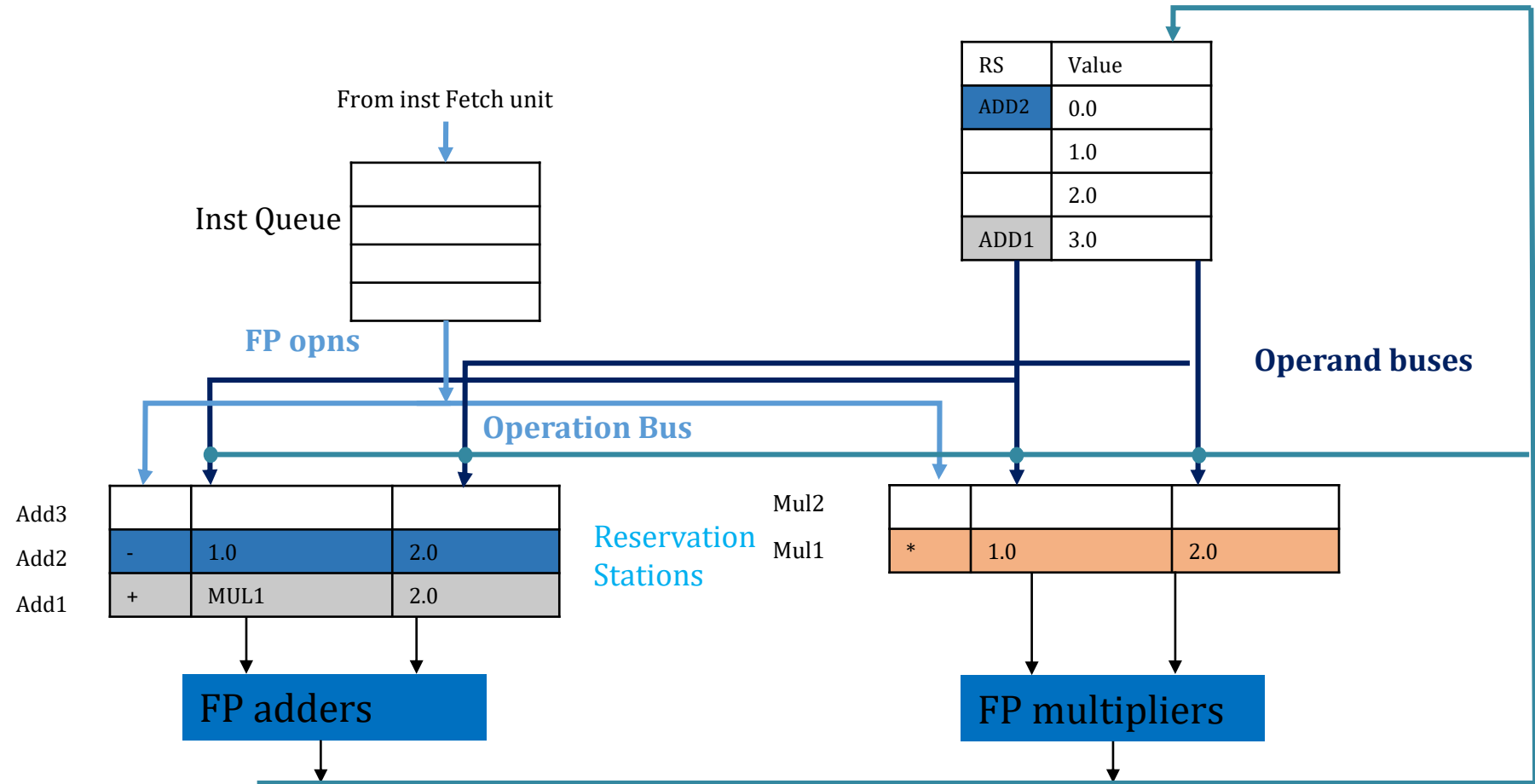


Register Renaming!

- ADD1 does not wait for F0 to become available
- RS1 Add1 waits for result Mul1 to be produced

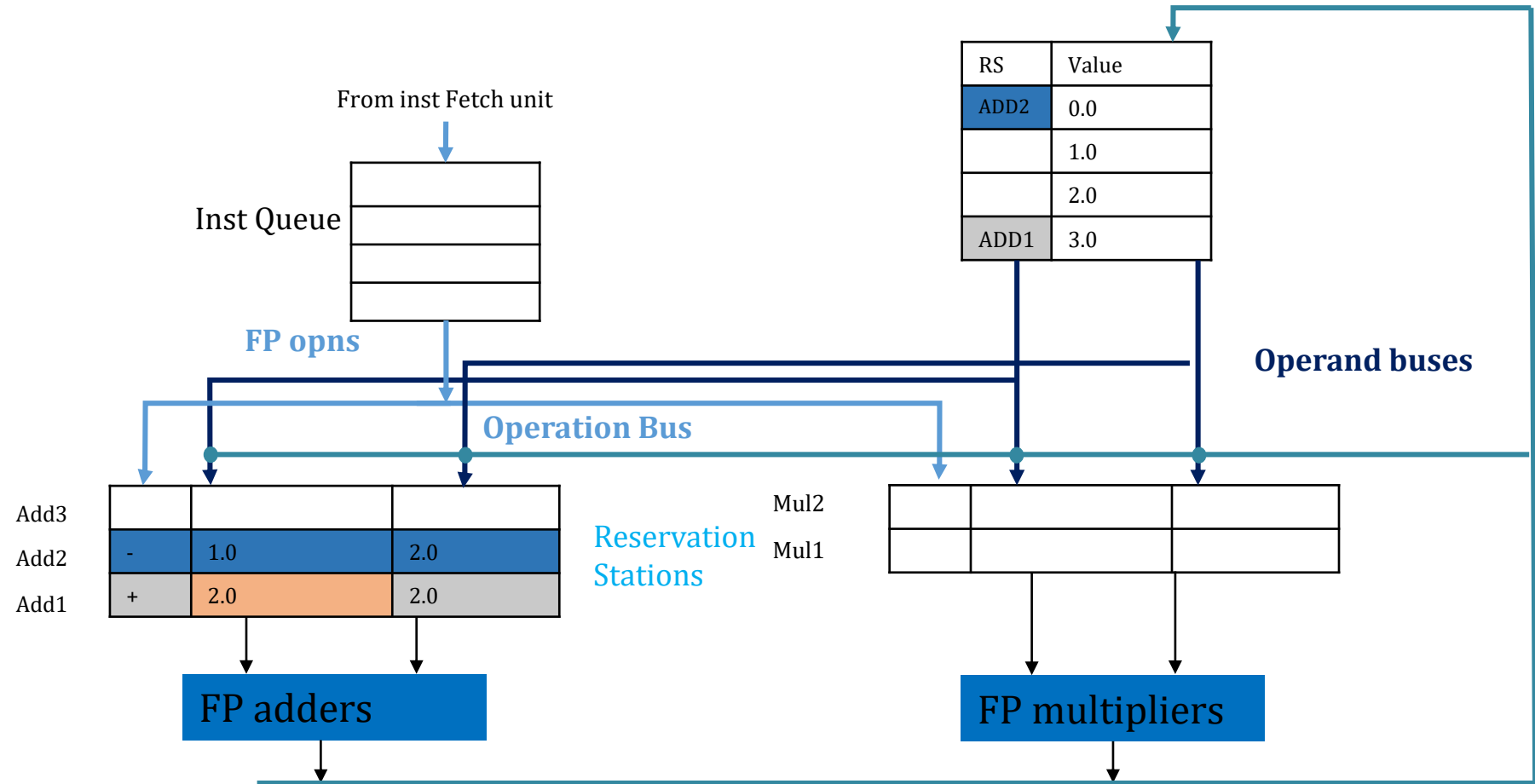
Issue SUB.D

MUL.D	F0, F1, F2
ADD.D	F3, F0, F2
SUB.D	F0, F1, F2



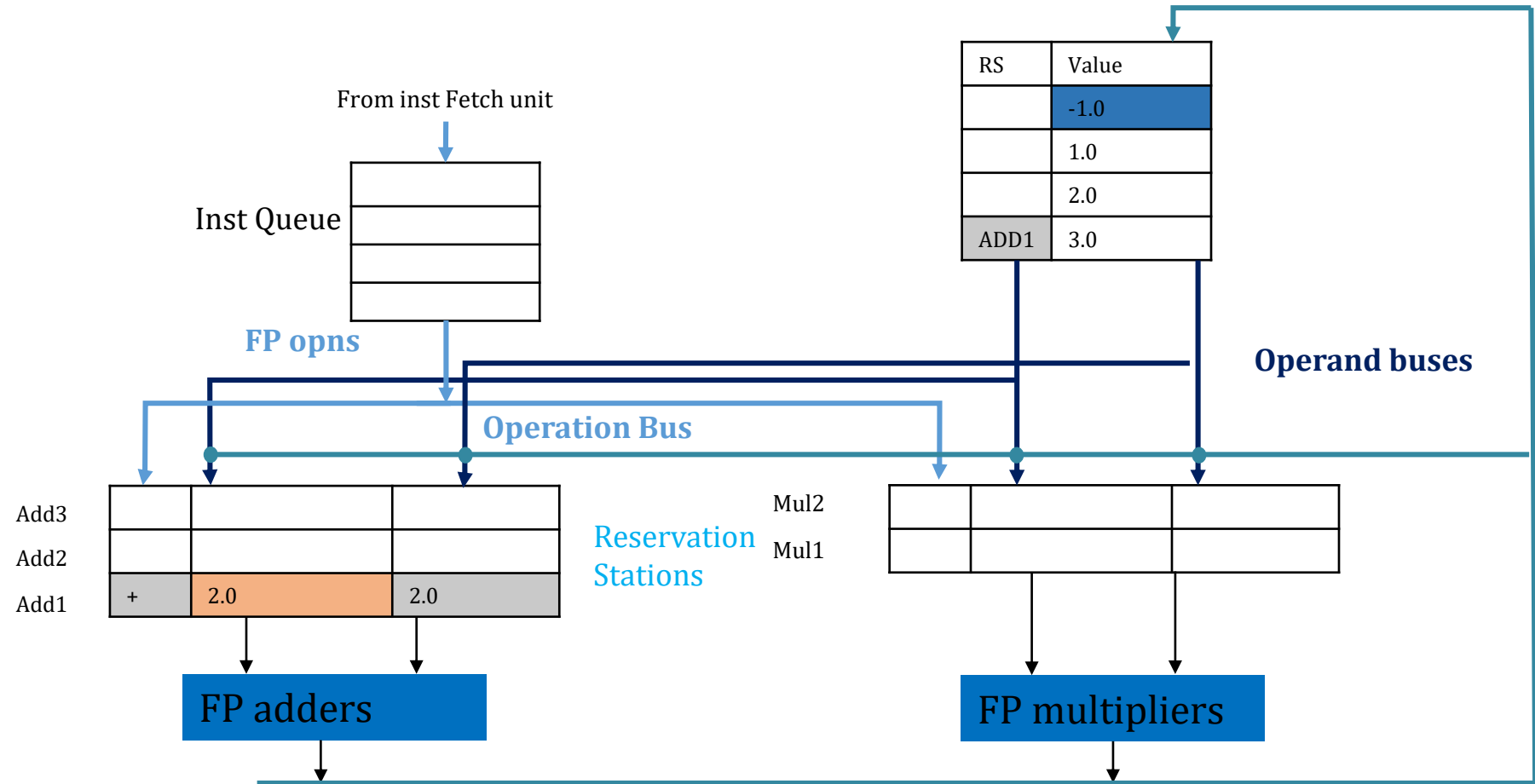
Write Result Mul1

MUL.D	F0, F1, F2
ADD.D	F3, F0, F2
SUB.D	F0, F1, F2



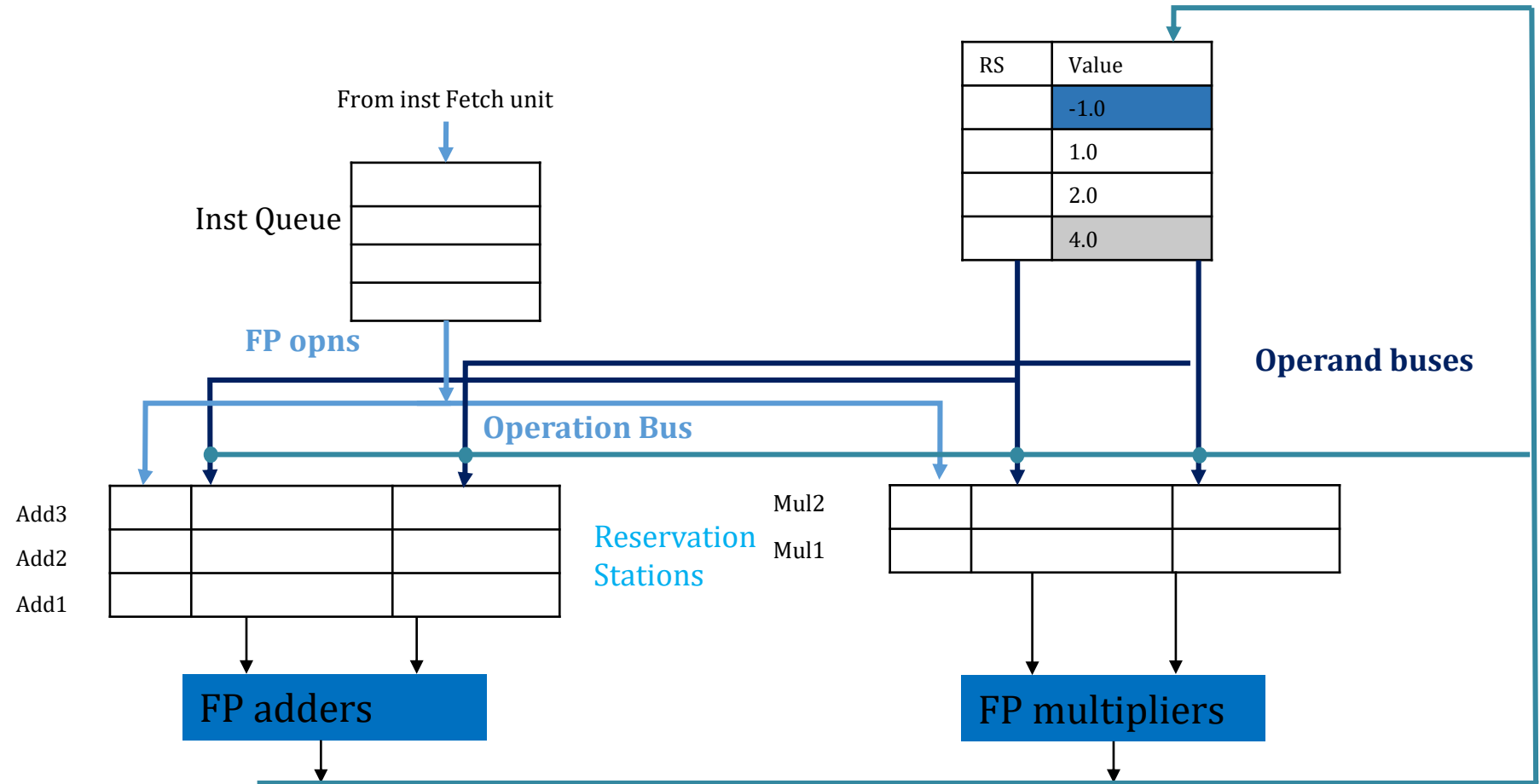
Write Result Add2

MUL.D	F0, F1, F2
ADD.D	F3, F0, F2
SUB.D	F0, F1, F2



Write Result Add1

MUL.D F0, F1, F2
ADD.D F3, F0, F2
SUB.D F0, F1, F2



Features of Tomasulo

- ❑ Register Renaming is done via Reservation Stations (RSs)
 - ❖ Elimination of WAR & WAW ($RS+RF$ = virtual register set)
 - ❖ RS has a distributed structure: Operand forwarded to multiple RSs in 1 cycle (centralize register file: sequential access is only 1 port).
- ❑ Bypassing/forwarding
 - ❖ via CDB, directly from execution unit to multiple RSs
- ❑ In-order issue/dispatch
- ❑ Out-of-order execution
- ❑ Out-of-order instruction retiring/completion (no precise exception)

Dynamic Scheduling of Memory Accesses

- ❑ Load and stores to different addresses can be executed out-of-order
- ❑ Load and stores accessing same address cause hazards

S.D F1, 1000(R0)
↓ RAW
L.D F0, 1000(R0)

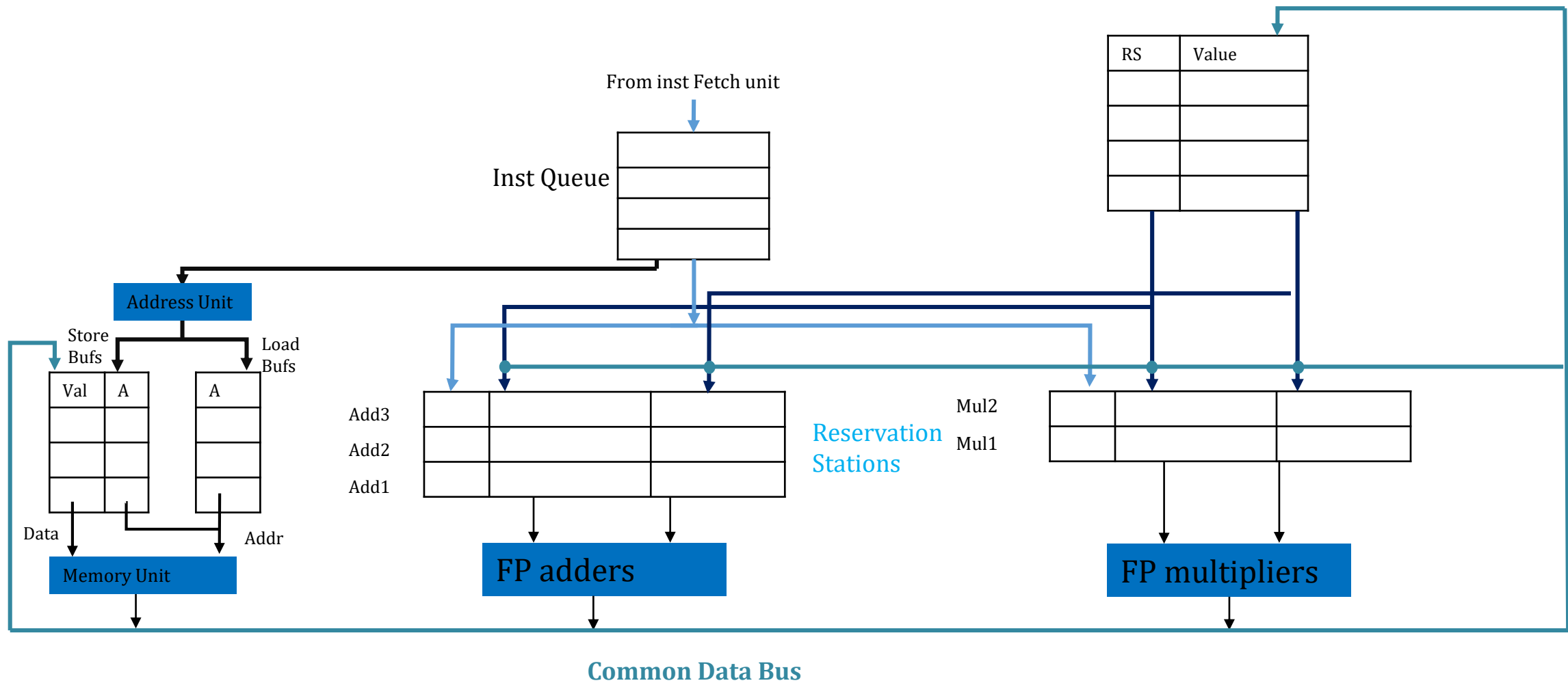
L.D F0, 1000(R0)
↓ WAR
S.D F1, 1000(R0)

S.D F1, 1000(R0)
↓ WAW
S.D F1, 1000(R0)

- ❑ Whether hazards occurs will be known only after calculating the effective address (EA)
 - ❖ Requires the EA of previous memory accesses

S.D F1, 1000(R0)
↓ RAW or no RAW?
L.D F0, 1000(R1)

Tomasulo-based FPU with memory accesses



Load and Store Execution

❑ Load and store consists of 2 steps

❖ EA calculations

❖ Memory Accesses

❑ Load and store buffers each have a field A

❖ EA calculated in 1 step

Store Buffers	
A	Val

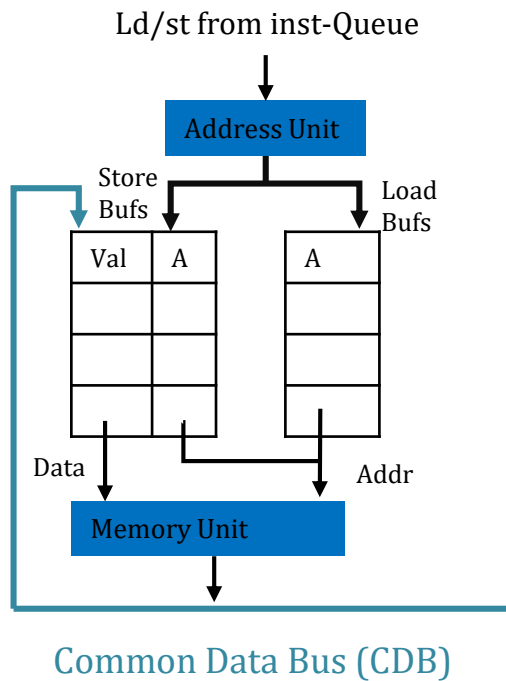
Load Buffers
A

❑ Store buffer Val field can also be a value produced by a RS

❑ RS can also be waiting for load buffer

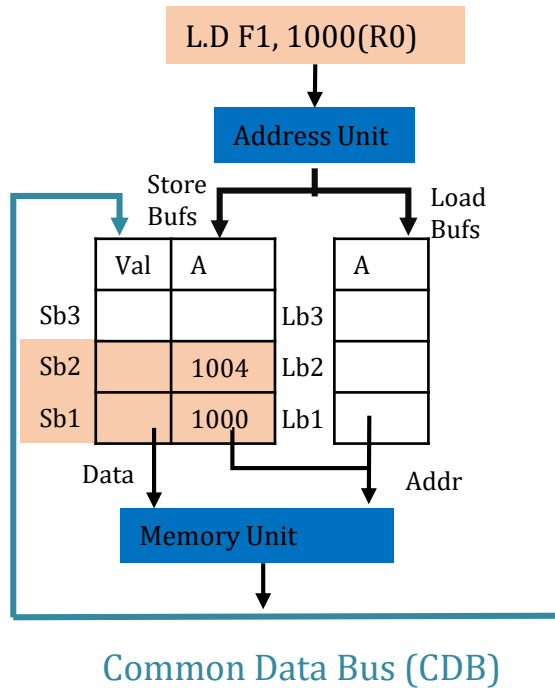
op	RS1	RS2	Val1	Val2	Imm/addr	busy
add	Lb2	0	n/a	12.55	n/a	1

Load and Store Issue



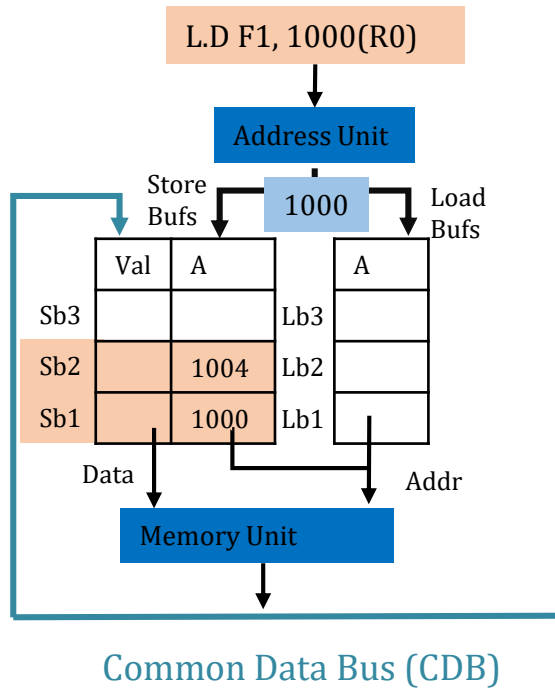
- Load
 - Address unit calculates EA
 - EA compared to A field of all active store bufs
 - If match (RAW), load not send to load buf until conflicting stores completes
- Store:
 - Similar but must check for conflict in both load bufs (WAR) and store bufs (WAW)

Load and Store Issue: Example



- Load
 - Address unit calculates EA
 - EA compared to A field of all active store bufs
 - If match (RAW), load not send to load buf until conflicting stores completes
- Store:
 - Similar but must check for conflict in both load bufs (WAR) and store bufs (WAW)

Load and Store Issue: Example



- Load
 - Address unit calculates EA
 - EA compared to A field of all active store bufs
 - If match (RAW), load not send to load buf until conflicting stores completes
- Store:
 - Similar but must check for conflict in both load bufs (WAR) and store bufs (WAW)

Example Loop

```
double x[1000], s;  
for (i=0; i<1000; i++)  
    x[i] = x[i]+s;
```

```
; R1 = x = &x[0]  
; R2 = &x[1000]  
; F2 = s  
for:    L.D      F0, 0(R1)      ; F0 =x[i]  
        ADD.D   F4, F0, F2     ; F4 = x[i] + s  
        S.D     F4, 0(R1)     ; x[i] = F4  
        DADDI   R1, R1, 8      ; R1 += 8 = &x[i+1]  
        BNE     R1, R2, for    ; if (R1!= &x[1000]) goto for
```


Assumptions of the hardware

- ❑ OoO execution

 - ❖ Integers as well as floating point (FP) instructions

- ❑ 1 integer functional unit (FU) (used for both ALU and EA calculations)

- ❑ Separate FU to evaluate branch conditions

- ❑ Separate pipeline FP FU

- ❑ Issue and Write take 1 cc each

- ❑ Single memory port

- ❑ Perfect branch prediction

 - ❖ Instructions are fetched and issued but not actually executed until branch has completed

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)					
1	ADD.D F4, F0, F2					
1	S.D F4, 0(R1)					
1	DADDI R1, R1, #8					
1	BNE R1,R2, for					
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2					
1	S.D F4, 0(R1)					
1	DADDI R1, R1, #8					
1	BNE R1,R2, for					
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2				
1	S.D F4, 0(R1)					
1	DADDI R1, R1, #8					
1	BNE R1,R2, for					
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5			
1	S.D F4, 0(R1)					
1	DADDI R1, R1, #8					
1	BNE R1,R2, for					
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					



Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)					
1	DADDI R1, R1, #8					
1	BNE R1,R2, for					
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					



Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4			
1	DADDI R1, R1, #8					
1	BNE R1,R2, for					
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8					
1	BNE R1,R2, for					
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					



Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for					
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5				
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)					
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					



Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6				
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8			Wait for BNE
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10		struct hazard
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2					
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2	7				
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2	7	12		15	Wait for L.D
2	S.D F4, 0(R1)					
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					



Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2	7	12		15	Wait for L.D
2	S.D F4, 0(R1)	8	9	16		Wait for ADD.D
2	DADDI R1, R1, #8					
2	BNE R1,R2, for					



Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2	7	12		15	Wait for L.D
2	S.D F4, 0(R1)	8	9	16		Wait for ADD.D
2	DADDI R1, R1, #8	9	10		12	
2	BNE R1,R2, for					

Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2	7	12		15	Wait for L.D
2	S.D F4, 0(R1)	8	9	16		Wait for ADD.D
2	DADDI R1, R1, #8	9	10		12	
2	BNE R1,R2, for	10	13			Wait for DADDI



Execution latencies:	Int ALU: 1 cc	Ld/st: 2 cc (EX+MEM)	FP add: 3cc
----------------------	---------------	----------------------	-------------

CPI of the Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2	7	12		15	Wait for L.D
2	S.D F4, 0(R1)	8	9	16		Wait for ADD.D
2	DADDI R1, R1, #8	9	10		12	
2	BNE R1,R2, for	10	13			Wait for DADDI

CPI of the Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2	7	12		15	Wait for L.D
2	S.D F4, 0(R1)	8	9	16		Wait for ADD.D
2	DADDI R1, R1, #8	9	10		12	
2	BNE R1,R2, for	10	13			Wait for DADDI

10 instructions in 16 cycles -> CPI = 1.6

CPI of the Dynamic Execution Example

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2	7	12		15	Wait for L.D
2	S.D F4, 0(R1)	8	9	16		Wait for ADD.D
2	DADDI R1, R1, #8	9	10		12	
2	BNE R1,R2, for	10	13			Wait for DADDI

10 instructions in 16 cycles -> CPI = 1.6

Unscheduled Loop on static pipeline = 2.0

Iter. #	Instructions	Issues @	Begins EX	Mem acc.	Write CDB	Comment
1	L.D F0, 0(R1)	1	2	3	4	First Issue
1	ADD.D F4, F0, F2	2	5		8	Wait for L.D
1	S.D F4, 0(R1)	3	4	9		Wait for ADD.D
1	DADDI R1, R1, #8	4	5		6	
1	BNE R1,R2, for	5	7			Wait for DADDI
2	L.D F0, 0(R1)	6	8	10	11	Wait BNE, struct hazards
2	ADD.D F4, F0, F2	7	12		15	Wait for L.D
2	S.D F4, 0(R1)	8	9	16		Wait for ADD.D
2	DADDI R1, R1, #8	9	10		12	
2	BNE R1,R2, for	10	13			Wait for DADDI
3	L.D F0, 0(R1)	11	14	15	16	Wait for BNE
3	ADD.D F4, F0, F2	12	17		20	Wait for L.D
3	S.D F4, 0(R1)	13	15	21		Wait for ADD.D
3	DADDI R1, R1, #8	14	16		17	
3	BNE R1, R2, for	15	18			Wait for DADDI

Normal Pipeline to Modern CPUs

- ❑ The 5-stage pipeline we have studied is the conceptual skeleton of every modern general-purpose CPU.
- ❑ Real designs add layers of sophistication but the same hazards and trade-offs remain.

1980s - Classic RISC	Single-issue, in-order, 5 stages	<i>CPI $\approx 1.1-1.3$</i>
1990s - Superscalar	2-4 issue, in-order, deeper pipelines	<i>CPI $\approx 0.6-0.8$</i>
2000s - Out-of-Order	Dynamic scheduling, register renaming, speculation	<i>CPI $\approx 0.3-0.5$</i>
2010s - Multicore	Many OoO cores, hardware multithreading, SIMD	<i>Throughput-oriented</i>
2020s - Heterogeneous	P/E cores, accelerators, on-die GPUs and NPUs	<i>Domain-specific perf/W</i>

Summary

❑ Performance is quantitative

- $\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{T}$. Every architectural decision should be evaluated by its measurable effect on these three factors and on power.

❑ ILP unlocks throughput

- Independent instructions can be executed in parallel. The architect's job is to find them and build hardware that exploits them.

❑ Pipelining overlaps execution

- Splitting an instruction into stages keeps every part of the datapath busy. In steady state, one instruction completes per cycle.

❑ Hazards limit efficiency

- Data, control, and structural hazards push effective CPI above its ideal value. Forwarding, branch prediction, and renaming mitigate them.

References

PRIMARY REFERENCE

Hennessy, J. L. and Patterson, D. A.

Computer Architecture: A Quantitative Approach - 6th Edition.

Morgan Kaufmann / Elsevier, 2017. ISBN 978-0-12-811905-1.

SUPPORTING REFERENCES

1. Patterson, D. A. and Waterman, A. - The RISC-V Reader: An Open Architecture Atlas. Strawberry Canyon, 2017.
2. Waterman, A. and Asanović, K. (Eds.) - The RISC-V Instruction Set Manual, Volume I: User-Level ISA. RISC-V Foundation, 2019.
3. Patterson, D. A. and Hennessy, J. L. - Computer Organization and Design: The Hardware/Software Interface (RISC-V Edition). Morgan Kaufmann, 2017.
4. Amdahl, G. M. - "Validity of the Single-Processor Approach to Achieving Large Scale Computing Capabilities." AFIPS, 1967.

Thank you